

ajedrezUPV — Documentacion del proyecto

Motor de ajedrez con inteligencia artificial

Desarrollado en la Universitat Politecnica de Valencia (UPV)
perezllorenc@gmail.com

August 19, 2026

Contents

1	Visión general	2
2	Estructura del repositorio	3
3	Motor de busqueda	3
4	Evaluador heuristico	4
5	Redes neuronales	5
6	Sistema de tests	5
7	Compilacion y ejecucion	6
8	Estado del proyecto	6
9	Licencia	7
10	Contacto y agradecimientos	7

1 Visión general

ajedrezUPV es un motor de ajedrez con inteligencia artificial, desarrollado en el marco de la Universitat Politecnica de Valencia (UPV). Combina algoritmos de búsqueda avanzados (alpha-beta con profundización iterativa, tabla de transposición, quiescence search, ordenación de movimientos) con redes neuronales para la evaluación de posiciones.

Características principales

- Motor de ajedrez en C++17 con la librería Disservin chess-library
- **Busqueda avanzada:** Alpha-Beta con profundización iterativa, aspiration windows, tabla de transposición (4-way set-associative), quiescence search con stand-pat, ordenación de movimientos (TT move, MVV-LVA, killers, history heuristic)
- **Evaluación heurística:** material con mapas de calor, estructura de peones (aislados, doblados, pasados), seguridad del rey (fracción de casillas vecinas bajo ataque enemigo), control del tablero, detección correcta de jaque mate y tablas
- **Redes neuronales** (C++): red grande ($2565 \rightarrow 2048 \rightarrow 1536 \rightarrow 1024 \rightarrow 768 \rightarrow 512 \rightarrow 256 \rightarrow 1$, $\sim 14\text{M}$ parámetros) y red junior ($782 \rightarrow 512 \rightarrow 512 \rightarrow 1$) con inicialización He, pérdida MSE, y serialización JSON/binaria
- **Entrenamiento PyTorch:** script con mini-batches, perspectiva del lado al mueve, objetivo $\tanh(cp/400)$, exportación a binario para inferencia C++
- Protocolo de juego interactivo con comandos: movimientos UCI, `undo`, `new`, `fen`, `d`, `go depth N`
- 52 tests automatizados (unitarios, perft, mate-in-N, end-to-end)

2 Estructura del repositorio

Ruta	Descripcion
CMakeLists.txt	Build CMake (con FetchContent para chess-library)
Makefile	Build Make
play.sh	Script para jugar facilmente
requirements.txt	Dependencias Python
include/chess/ evaluator.h general_evaluator.h opening_evaluator.h endgame_evaluator.h node_move_optimized.h transposition_table.h search.h	Cabeceras publicas del motor Interfaz base de evaluacion + constantes (<code>eval::</code> namespace) Evaluador general con metodos comunes Evaluador de apertura (mapas de calor) Evaluador de finales (rey centralizado) Nodo del arbol + SearchResult + quiescence (legacy) Tabla de transposicion (4-way set-associative) Busqueda on-demand con TT + ID + aspiration windows
src/engine/ alphabeta_optimized.cpp node_move_optimized.cpp transposition_table.cpp search.cpp evaluation/	Motor de ajedrez Motor principal (bucle de juego, comandos) Arbol optimizado con alpha-beta + quiescence (legacy) Implementacion de la tabla de transposicion Busqueda on-demand (TT + ID + quiescence + killers) Evaluadores heuristicos (general, apertura, final)
src/neural/ network_junior.hpp square_rook_network.cpp simd_network_wip.cpp	Redes neuronales C++ Red junior (782→512→512→1) Red grande (~14M parametros) Prototipo SIMD (AVX2, WIP)
src/tools/ json_extractor.cpp json_extractor_junior.cpp	Herramientas de datos Extractor de evaluaciones JSONL Extractor + entrenador inline con perspectiva
python/ train.py model.py network_reference.py mnist_loader.py	Herramientas Python Entrenamiento PyTorch (mini-batches, tanh, export binario) Análisis de datasets y visualizacion Referencia de red neuronal (Michael Nielsen) Cargador MNIST (referencia)
test/ data/dataset.csv lichess_db/ third_party/ docs/ LICENSE	52 tests automatizados Dataset de analisis Chunks JSONL de evaluaciones de Lichess json.hpp, chess.hpp Documentacion LaTeX y PDFs CC BY-NC 4.0

3 Motor de busqueda

El motor utiliza dos implementaciones de busqueda:

Search (recomendada)

Busqueda on-demand con generacion lazy de movimientos. Implementa:

- Profundizacion iterativa con aspiration windows ($\text{depth} \geq 4$)
- Tabla de transposicion 4-way set-associative (16 MB por defecto)
- Quiescence search con stand-pat y MVV-LVA
- Ordenacion: TT move → capturas MVV-LVA → promociones → killers → history
- Deteccion de jaque mate, ahogado, tablas (50 movimientos, repeticion, material insuficiente)

- Control de tiempo para futura integracion UCI

NodeMoveOptimized (legacy)

Arbol pre-construido con arena PMR (`std::pmr::unsynchronized_pool_resource`). Mismas tecnicas de busqueda pero con el arbol materializado en memoria. Util para depuracion y benchmarks.

Tabla de transposicion

- Entradas: hash (64 bits) + movimiento + puntuacion + profundidad + flag + edad
- Buckets de 4 entradas (set-associative)
- Flags: EXACT, LOWER (beta-corte), UPPER (fail-low)
- Puntuaciones de mate ajustadas por ply (`scoreForTT/scoreFromTT`)

Constantes de busqueda

```
MATE_SCORE   = 30000.0f    // Puntuacion de jaque mate
INFINITY      = 50000.0f    // Infinito de la ventana
MAX_DEPTH     = 3           // Profundidad maxima (arbol legacy)
MAX_BRANCH    = 100         // Limite de hijos por nodo (legacy)
```

4 Evaluador heuristico

Escala

Todos los valores estan en centipeones:

```
PAWN_VALUE    = 100      KNIGHT_VALUE = 300      BISHOP_VALUE = 315
ROOK_VALUE    = 500      QUEEN_VALUE  = 900
```

Fases del juego

La evaluacion detecta la fase por presencia de damas:

- **Apertura** (`OpeningEvaluator`): mapas de calor para piezas menores, rey en esquinas
- **Final** (`EndgameEvaluator`): rey centralizado, peones avanzados

Terminos de evaluacion

1. **Material + posicion**: suma de valores base + mapas de calor por pieza
2. **Estructura de peones**: penalti por peones aislados (-0.5), doblados (-0.5), bonus por pasados ($+1.0$). Comprueba todas las filas hacia adelante.
3. **Seguridad del rey**: fraccion de casillas vecinas al rey bajo control enemigo (0 = seguro, 1 = inseguro)
4. **Control del tablero**: casillas controladas por el color consultado, pesadas por importancia central

Perspectiva

Todos los terminos devuelven puntuacion en perspectiva del *lado al mueve* (convencion negamax). La busqueda niega el signo al subir por el arbol.

5 Redes neuronales

Red junior (`network_junior.hpp`)

- Arquitectura: $782 \rightarrow 512 \rightarrow 512 \rightarrow 1$
- Inicializacion He: $\text{weights} \sim \mathcal{N}(0, \sqrt{2/N_{in}})$
- Activacion: ReLU (capas ocultas), lineal (salida)
- Perdida: MSE ($\frac{1}{2}(\text{pred} - \text{target})^2$)
- Optimizador: Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$)

Red grande (`square_rook_network.cpp`)

- Arquitectura: $2565 \rightarrow 2048 \rightarrow 1536 \rightarrow 1024 \rightarrow 768 \rightarrow 512 \rightarrow 256 \rightarrow 1$
- ~ 14 millones de parametros
- Perdida: Huber (con $\text{delta}=1.0$)
- Serializacion a JSON con `std::async` para paralelismo

Encoding de entrada (782 features)

1. 768 entradas: 12 tipos de pieza $\times$ 64 casillas (one-hot)
2. 4 entradas: derechos de enroque
3. 1 entrada: lado al mueve (siempre 1.0, codificado desde nuestra perspectiva)
4. 8 entradas: casilla de captura al paso (en passant)
5. 1 entrada: reloj de 50 movimientos (normalizado)

Cuando mueve negro, el tablero se espeja verticalmente para que “nuestras” piezas siempre avancen “hacia arriba”. Esto reduce a la mitad los patrones que la red debe aprender.

Entrenamiento PyTorch (`python/train.py`)

- Mini-batches de 256 ejemplos
- Funcion objetivo: $\tanh(cp/400) \in [-1, 1]$
- Weight decay (10^{-4}) en lugar de dropout
- Seeds fijos para reproducibilidad
- Exportacion a tres formatos: `.pt` (PyTorch), `.bin` (binario float32), `.json`

6 Sistema de tests

El proyecto incluye 52 tests automatizados organizados en 6 suites:

Suite	Tests	Que verifica
<code>test_node_move</code>	3	Arbol de nodos, rebuild, alpha-beta con restauracion de tablero
<code>test_search_benchmark</code>	1	Benchmark de busqueda depth 1-3
<code>test_json_extractor</code>	3	Parser de evaluaciones JSONL de Lichess
<code>test_json_extractor_junior</code>	3	Conversion FEN a tensor de entrada (782 features)
<code>test_phase2</code>	13	TT, busqueda on-demand, perft, mate-in-1 y mate-in-2
<code>test_e2e</code>	29	End-to-end: arranque, movimientos, EOF, comandos, reglas de ajedrez

Tests destacados

- **Perft**: validacion de generacion de movimientos en posiciones conocidas (startpos, kiwipete, position3) con nodos por profundidad exactos
- **Mate-in-1**: jaque mate por la espalda (Re8#), jaque mate del erudito (Qxf7#)
- **Mate-in-2**: patron de Damiano con busqueda a profundidad 6
- **EOF critico**: verifica que el motor no entra en bucle infinito al cerrar stdin
- **Tablero restaurado**: verifica que la busqueda no altera el tablero de entrada

7 Compilacion y ejecucion

Con Makefile

```
make all          # Compila motor + herramientas neuronales
make engine       # Solo el motor
make neural       # Solo herramientas neuronales
make test         # Compila y ejecuta los 52 tests
make clean        # Limpia binarios
```

Con CMake (recomendado)

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j$(nproc)
cd build && ctest
```

Jugar

```
./play.sh          # Profundidad 3
./play.sh --depth 5 # Profundidad 5
./play.sh --fen "FEN" # Desde una posicion concreta
./play.sh --build   # Compilar primero
./bin/alphabeta_optimized # Directamente
```

Comandos durante la partida

```
e2e4      Mover pieza (notacion UCI)
e7e8q     Promocion de peon
e8g8      Enroque corto
d         Mostrar tablero
fen       Mostrar FEN actual
undo      Deshacer 2 movimientos (tuyo + del motor)
new       Nueva partida
go depth 5 Motor juega con profundidad 5
quit      Salir
```

8 Estado del proyecto

Completado

- Motor con Alpha-Beta + profundizacion iterativa + tabla de transposicion
- Quiescence search + ordenacion de movimientos (TT move, MVV-LVA, killers, history)
- Evaluacion heuristica (material, peones, seguridad rey, control)

- Deteccion correcta de jaque mate, ahogado y tablas
- Interfaz con comandos (quit, undo, new, fen, d, go depth)
- 52 tests automatizados (unitarios, perft, mate-in-N, end-to-end)
- Script `play.sh` para jugar facilmente
- Redes neuronales C++ (junior y grande) con inicializacion He
- Entrenamiento PyTorch con mini-batches y exportacion binaria
- Perspectiva del lado al mueve en encoding y evaluacion

Pendiente

- Podas avanzadas (null-move pruning, LMR, futility)
- NeuralEvaluator integrado como **Evaluator**
- Protocolo UCI completo
- Multi-hilo (Lazy SMP)
- Libro de aperturas
- Medicion de fuerza con cuteshess-cli
- Arquitectura NNUE (acumulador incremental)

9 Licencia

Este proyecto esta licenciado bajo la Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0).

- **Permitido:** compartir, adaptar, uso educacional/personal/investigacion
- **No permitido:** uso comercial sin permiso explicito del autor

10 Contacto y agradecimientos

Autor: Llorenc Perez — perezlllorenc@gmail.com

Repositorio: <https://github.com/LLPB-pixel/ajedrezUPV>

Agradecimientos: UPV, Disservin (chess-library), nlohmann (json), Lichess (base de datos de evaluaciones), PyTorch team.

Generado: August 19, 2026